

MINI PROJECT REPORT

SUBMITTED BY: MEENAKSHI VEDALA (PRN: 24070721045)

Overview:

The **Student Performance Analyzer (SPA)** is a secure, role-based terminal application developed using **Python 3** and **MySQL**, designed to manage and analyze student academic records. The system authenticates users via SHA-256 password hashing and enforces three distinct access roles — Admin, Faculty, and Student — each presented with a menu restricted to their permitted operations. The underlying database comprises three relational tables: users for credentials, students for academic records, and audit_log for timestamped tracking of every data-modifying event. Core features include full student CRUD operations, a live analytics dashboard with grade distributions and department-wise statistics, bonus marks allocation, CSV export of high performers, and a complete user management system. All database interactions are executed through parameterized queries to prevent SQL injection, and every operation is wrapped in structured exception handling to ensure application stability.

Key Design Choices:

1. **SHA-256 Password Hashing** — All passwords are hashed using Python's `hashlib.sha256` before storage and comparison. No plain-text passwords are ever written to the database, and `getpass.getpass()` is used for all password inputs to prevent terminal exposure.
2. **Parameterized Queries** — Every SQL statement uses `%s` placeholders with `mysql.connector`'s `parameterized.execute()` method, eliminating any risk of SQL injection regardless of user input.
3. **Role-Based Menu Dispatch** — After login, the user's role drives which menu function is called. Each menu is defined as a dictionary mapping option keys to function references, making it straightforward to manage permissions per role without affecting other menus.
4. **UTF-8 Collation** — The database and username column both use `utf8mb4_unicode_ci`, allowing MySQL to perform case-insensitive username comparisons at the database level without requiring a `LOWER()` wrapper on every query.
5. **LEAST() for Bonus Marks Cap** — Bonus marks are applied using `UPDATE students SET marks = LEAST(marks + %s, 100)`, pushing the 100-cap logic into MySQL itself rather than fetching, computing, and re-writing values in Python, making the operation more efficient and atomic.
6. **Fault-Tolerant Audit Logging** — The `log_audit()` function wraps its own database call in an independent `try/except` block, ensuring that an audit failure can never crash the main application.
7. **Structured Exception Handling** — Every database operation is wrapped in `try/except/finally` blocks that close the cursor and connection in the `finally` clause regardless of whether an error occurred, preventing connection leaks.

Challenges and Solutions:

1. **Audit Log Independence** — Ensuring the audit log never interfered with the main application was a key challenge. If the logging function raised an exception, it could potentially crash an otherwise successful operation. This was solved by wrapping `log_audit()` in its own isolated `try/except` block, completely decoupled from the calling function's error handling.
2. **Role Enforcement Without a Framework** — Without a web framework's built-in authentication middleware, enforcing role restrictions in a terminal application required a deliberate approach. This was solved using a global `current_user` dictionary populated at login, with each function checking the role before executing any logic, replicating session-based access control in plain Python.
3. **Bonus Marks Atomicity** — Applying bonus marks to an entire department required ensuring that either all records were updated or none were, with no partial updates left in the database. This was handled by relying on MySQL's transactional behaviour with a single `UPDATE` statement using `LEAST(marks + %s, 100)`, making the entire operation atomic.
4. **Preventing Duplicate Usernames** — When creating new user accounts, duplicate usernames needed to be rejected gracefully rather than throwing a raw database error. This was handled by relying on the `UNIQUE` constraint defined on the username column, catching the resulting `IntegrityError` in Python and displaying a clean, user-friendly message instead.
5. **Linking Student Records to User Accounts** — When an Admin creates a student-role user, two separate `INSERT` operations are required — one into `users` and one into `students` — with the new user's ID passed as a foreign key. The challenge was ensuring both inserts succeeded together. This was handled by retrieving `cursor.lastrowid` immediately after the first `INSERT` and using it in the second, keeping both records consistent.

Areas where AI was used:

Artificial Intelligence was utilised throughout this project as a development accelerator, enabling a higher standard of implementation than would have been achievable within the given timeframe alone.

- Code Architecture & Structuring** — AI was used to translate the functional logic of the application into a clean, modular Python structure. Rather than writing monolithic code, AI assisted in organising the project into well-defined functions with clear separation of concerns, consistent error handling patterns, and role-based menu dispatch — producing professional, maintainable code aligned with industry standards.
- SQL Schema Design** — AI assisted in designing a robust database schema incorporating PRIMARY KEY, FOREIGN KEY, CHECK constraints, and UTF-8 Unicode collation. This ensured referential integrity, input validation, and case-insensitive matching were enforced at the database level rather than relying solely on application logic.
- Security Implementation** — AI provided guidance on best practices for password security, recommending SHA-256 hashing via hashlib and silent password input via getpass.getpass(). This elevated the application beyond a basic student project to one that follows real-world authentication standards.
- Debugging & Problem Solving** — When runtime errors were encountered, such as the Windows UnicodeEncodeError caused by terminal encoding incompatibilities, AI rapidly diagnosed the root cause and provided a targeted fix, significantly reducing debugging time.

Output and Testing:

id	name	department	marks	user_id	enrolled_at
1	Alice Johnson	CSE	92.00	3	2026-05-12 10:39:50
2	Bob Smith	CST	78.50	4	2026-05-12 10:39:50
3	Carol White	CSE	65.00	5	2026-05-12 10:39:50
4	Dave Brown	ECE	45.50	6	2026-05-12 10:39:50
7	Arjun Reddy	CSE	72.00	NULL	2026-05-12 12:21:50
8	Meena Pillai	CST	61.50	NULL	2026-05-12 12:21:50
9	Suresh Babu	ECE	45.00	NULL	2026-05-12 12:21:50
10	Lakshmi Devi	ECE	55.00	NULL	2026-05-12 12:21:50
11	Kiran Rao	CSE	91.00	NULL	2026-05-12 12:21:50
12	Divya Nair	CST	38.00	NULL	2026-05-12 12:21:50
13	Anil Teja	ECE	82.50	NULL	2026-05-12 12:21:50
14	Sneha Varma	CSE	67.00	NULL	2026-05-12 12:21:50

```
mysql> select* from users;
```

id	username	password_hash	role	is_active	created_at
1	admin	240be518fabd2724dd6f04eeb1da5967448d7e831c88c8fa822889f74c720a9	admin	1	2026-05-12 10:39:48
2	faculty1	27041f5856c7387a997252694afb048d1aa939228ffcd6d6285b979b8da20e7a	faculty	1	2026-05-12 10:39:48
3	alice	4e40e8ffe0ee32fa53e139147ed559229a5930f89c2204706fc174beb36210b3	student	1	2026-05-12 10:39:48
4	bob	84859c36d0b9718dd2ee453e28d391ab8cbf2eccbe87d01915a8e578a282511	student	1	2026-05-12 10:39:48
5	carol	686b75107c6da94929779bf2859c36cc95f9488270413a324ed09f94c71bd	student	1	2026-05-12 10:39:48
6	dave	d84201a2ccf0a519d5da6c003e8a24a73fae5982eac25b69ef0c994d9e34b88c	student	1	2026-05-12 10:39:48

```
mysql> select * from audit_log;
```

id	user_id	action	details	timestamp
1	NULL	LOGIN_FAILED	user failed login	2026-05-12 10:58:22
2	NULL	LOGIN_FAILED	user failed login	2026-05-12 10:58:31
3	NULL	LOGIN_FAILED	user failed login	2026-05-12 10:59:22
4	NULL	LOGIN_FAILED	admin123 failed login	2026-05-12 11:00:05
5	NULL	LOGIN_FAILED	admin123 failed login	2026-05-12 11:00:26
6	NULL	LOGIN_FAILED	admin123 failed login	2026-05-12 11:01:35
7	1	LOGIN	admin logged in	2026-05-12 11:02:09
8	1	LOGIN	admin logged in successfully	2026-05-12 11:56:00